

COL 331 (OPERATING SYSTEMS)

Assignment 1

Shivam Singh
2020CS10383

II. System calls

TRACE_ON and TRACE_OFF:

Defined two states TRACE_ON as 1 and TRACE_OFF as 0 in trace.h file.
And a global variable "trace" keeps the track of the tracing state.

```
#define TRACE_ON 1
#define TRACE_OFF 0
```

"trace.h"

sys_toggle():

sys_toggle() system call defined in "sysproc.c" file and all the related changes have been made in "user.h", "syscall.h", "syscall.c", "usys.S" and "proc.h" files. This system call basically changes the state of "trace" from TRACE_ON to TRACE_OFF or TRACE_OFF to TRACE_ON as per the current state and at the same time we have to reset one array which keeps the count of system calls at the invoke of TRACE_ON.

```
extern int trace;
extern int CALLS[ONSC+1];
extern char *Call_name[];
```

"trace.h"

```
int print_count(void);
int add(int,int);
int ps(void);
int send(int, int, void*);
int recv(void*);
int send_multi(int,int*,void*);
```

"user.h"

```
#define SYS_toggle 22
#define SYS_print_count 23
#define SYS_add 24
#define SYS_ps 25
#define SYS_send 26
#define SYS_recv 27
#define SYS_send_multi 28
```

"syscall.h"

```
extern int sys_toggle(void);
extern int sys_print_count(void);
extern int sys_add(void);
extern int sys_ps(void);
extern int sys_send(void);
extern int sys_recv(void);
extern int sys_send_multi(void);
```

"syscall.c"

```

[SYS_toggle] sys_toggle,
[SYS_print_count] sys_print_count,
[SYS_add] sys_add,
[SYS_ps] sys_ps,
[SYS_send] sys_send,
[SYS_recv] sys_recv,
[SYS_send_multi] sys_send_multi,
};

```

“syscall.c”

```

SYSCALL(toggle)
SYSCALL(print_count);
SYSCALL(add);
SYSCALL(ps);
SYSCALL(send);
SYSCALL(recv);
SYSCALL(send_multi);

```

“usys.S”

```

int
sys_toggle(void){
    if(trace==TRACE_ON){
        trace = TRACE_OFF;
    }
    else{
        trace = TRACE_ON;
        for(int i=1;i<=ONSC;i++){
            CALLS[i]=0;
        }
    }
    return trace;
}

```

“sysproc.c”

sys_print_count():

This system call is defined in “sysproc.c” file and all the related changes have been made the same as in the sys_toggle() system call. For implementing the count for the system calls I have created an “array” in “trace.h” file and this array’s element is incremented on the invoke of any system call and this happen in “syscall” function in “syscall.c” file. On the call of sys_print_count() system call all this array is printed on the terminal mapped with the name of system call in alphabetically sorted order.

```

int
sys_print_count(void){
    for(int i=1;i<=ONSC;i++){
        if(CALLS[map[i]]>0){
            cprintf(Call_name[map[i]]);
            cprintf(" ");
            cprintf("%d\n",CALLS[map[i]]);
        }
    }
    return 0;
}

```

”sysproc.c”

sys_add():

This system call is defined in “sysproc.c” file and all the necessary changes required in previously mentioned files have been made. This system call takes two int arguments a and b and returns a+b. For taking int argument I have used argint(int,int*) function.

```
int
sys_add(){
    int a,b;
    argint(0,&a);
    argint(1,&b);
    return a+b;
}
```

“sysproc.c”

sys_ps():

This system call is defined in “sysproc.c” file and along with all the previously mentioned changes new changes have been made in “defs.h” file and definition for a user function “ps()” has been made in “proc.c” file. This system call on being invoked makes the list of all the running processes on the system along with the name of the processes.

```
int
sys_ps(void){
    ps();
    return 0;
}
```

“sysproc.c”

```
int
ps(void){
    struct proc *p;
    for(p=ptable.proc;p<&ptable.proc[NPROC];p++){
        if(p->state!=UNUSED)printf("%5d %s\n",p->pid,p->name);
    }
    return 0;
}
```

“proc.c”

III. Inter-Process Communication

In this section of the assignment three system calls have been designed namely:

```
int sys_send(int sender_pid, int rec_pid, void *msg)
int sys_recv(void *msg)
```

```
int sys_send_multi(int sender_pid, int rec_pids[], void *msg)
```

```
char buffers[BUFFER_SIZE][8]={ "      "};  
int sender_pids[BUFFER_SIZE]={0};  
int rcv_pids[BUFFER_SIZE]={0};  
int queue[NPROC][MAX_QUEUE_SIZE]={0};  
  
void enqueue(int pid, int index,void* ch)  
  
int deque(void *msg,int pid)  
  
"comm.h"
```

Now lets have a look on the implementation and working of all these functions:

sys_send(int, int, void*):

This system call on invoking sends a message not more than size “8 bytes” from one process to another process. For this purpose it takes two “pids” (one for sender and one for receiver) and a “void pointer” to the message that needs to be transferred. For implementing the mechanism of send we need a “global buffer” (here I have taken a buffer of size 1000) and each buffer element is a character array of size 8, a array implementation of queue per process and necessary functions for to maintain the queue (namely enqueue and deque). Now whenever there is a sys_send() system call we enqueue the message in the queue of the receiving process and also store the sender’s pid corresponding to that message. Instances of “struct spinlock” have been used adequately as there might be more than one processes accessing the same buffer at the same time. Using lock helps in handling “race conditions”.

sys_rcv(void*):

On the invoke of this system call, if the queue corresponding to the calling process is non-empty then it emplace the content of the queue top in the memory location whose void pointer is passed as argument in this system call and then deque the queue. And in the case in which the queue is empty it waits till the time any other process sends it any message. Instances of “struct spinlock” have been used adequately as there might be more than one processes accessing the same buffer at the same time. Using locks helps in handling “race condition”.

sys_send_multi(int, int*, void*):

This system call is used to broadcast a message to multiple processes in a single call. The function takes as parameter an integer (pid of sender), an int array (array of pids of receiving processes) and a void pointer to the message that need to be broadcasted. For the implementation of this mechanism I have used “sys_send” syscall as a helper function i.e., I have iterated through the receiving processes pid array and call the system call “sys_call” once for all of them.

Definition of Unicast and Multicast communication and how they can be made to happen using above defined system call:

Unicast: When the communication is one on one there is only one sender and only one receiver and a msg that has to be sent.

Multicast: The communication is broadcasting type communication in which there is only one sender but more than one receiver to see the same message.

For **Unicast** we will call **send()** system call.

And for **multicast** we will be using **multi_send()** system call.

IV. Distributed Algorithm

For the implementation of the “SUM” function:

Initially there is only one process on the run of main and we have an array of size 1000 on which sum has to be computed. For distributing the task I have made 7 other child processes using **fork()** function and divided the array which is globally known by all the processes into 8 chunks and send the starting and ending point of the chunks to their respective solver processes using **send() syscall**. Now each process computes the partial sum of its corresponding chunk and sends the sum to the parent process using **send() syscall** (note: one parent process and all remaining is its child) and call **exit()**. In return the parent process receives the partial sums and call **wait()** and adds up all the partial sums to get the final sum of the array.

For the implementation of the “Variance” function:

By the previous mechanism we found the sum of the array and now we need the mean of the array which we can get by dividing the sum by 1000. Now we again create 7 other processes by using **fork()** system call and send all mean to all the children processes by using **multi_send()** system call for broadcasting

the mean to all the children processes. On the other hand all the children processes will call **recv()** system call for receiving the message sent by the parent broadcaster by this communication all processes have mean. Now all processes need to know in what chunk of the array they have to perform sum of the squares of the differences about the mean, for this parent process need to send the starting and ending index of the chunks of array to the all the children processes (the array is divided into 8 chunks of 125 length and each chunks are as follow (0 to 124),(125 to 249),, (875 to 999)). This information is sent by the parent process to the children process by using **send()** system call. After that each process performs its computation and sends the result to the parent by **send()** system call and then **exit()**. Parent receives the messages sent by all the children, calls **wait()** 7 times and then sums up all the value and divide it by 1000 to get the variance.
